UNIVERSIDADE FEDERAL DE SERGIPE
DEPARTAMENTO DE COMPUTAÇÃO

DISCIPLINA: Inteligência Artificial

DOCENTE: Hendrik

AVALIAÇÃO 4 - Chatbot com memória

PARTICIPANTES:

LUAN ALMEIDA VALENÇA

LUIZ FELIPE TOJAL GOMES CORUMBA

RAFAEL MACHADO COSTA MENESES

RAFAEL SANTOS SILVA

SÃO CRISTÓVÃO – SE

02/2026

1. Descrição do Problema

O problema a ser solucionado é a criação de um chatbot alimentado por LLM 's como Gemini. O ponto principal do problema é a persistência do histórico na conversa, para que o chatbot consiga continuar conversas que foram rodadas em outras execuções.

2. Large Language Models (LLMs)

Os LLM's são sistemas de inteligência artificial capazes de processar sequências de texto de maneira probabilística, aprendendo padrões e dependências contextuais para produzir respostas coerentes. Eles utilizam *embeddings*, que transformam palavras e frases em vetores numéricos, permitindo capturar relações semânticas e sintáticas de forma precisa.

Esses embeddings são então processados por *Transformers*, uma arquitetura baseada em camadas de atenção. O mecanismo de atenção, especialmente o *self-attention*, é o núcleo desses modelos: ele avalia a importância de cada palavra em relação às demais dentro de uma sequência, o que possibilita lidar com contextos longos e complexos. Essa combinação de *embeddings* e *Transformers* torna os LLMs adequados para tarefas como tradução automática, geração de texto, análise de sentimentos e sistemas de diálogo.

3. Solução do Problema

A solução implementada consiste em uma arquitetura modular que integra modelos preexistentes na nossa aplicação *streamlit*, e assim nós utilizamos essas abordagens para completar o chatbot:

1. Persistência via JSON: O histórico de interações é serializado e armazenado em arquivos JSON estruturados. Cada sessão recebe um *id* único, permitindo que o estado da conversa e as configurações do modelo sejam recarregados em execuções futuras, superando a volatilidade da memória. Além disso, eles são nomeados de acordo com o conteúdo da primeira mensagem.
2. Gerenciamento de Contexto, *Sliding Window*: Para otimizar o consumo de tokens e respeitar os limites operacionais das APIs, foi implementada uma técnica de janela deslizante (Sliding Window). O sistema trunca o histórico dinamicamente, mantendo apenas um certo número de mensagens mais recentes, garantindo que o núcleo do contexto atual seja enviado sem exceder a capacidade do modelo.
3. Realimentação e Orquestração de Requests: O fluxo de dados opera por realimentação contínua, onde cada entrada do usuário é apensada ao histórico recuperado e enviada via requisições HTTP POST para o endpoint do Gemini. A solução trata os retornos JSON para extração e exibição da resposta ao usuário.

4. Pseudocódigo e Implementação

Abaixo, detalhamos onde cada etapa do pseudocódigo se manifesta no código:

A. Tabela do Pseudocódigo

Pseudocódigo	Implementação
function CONVERSATIONAL-AGENT(user_input, history, config)	def conversational_agent_multimodal(user_input, image_path, history, config, api_key):
current_msg = CREATE-MESSAGE(role="user", content=user_input)	current_msg = Message(role="user", content=content)
APPEND(history, current_msg)	history.append(current_msg)
payload = TRUNCATE-HISTORY(history, limit=config.max_tokens)	payload_msgs = TRUNCATE_HISTORY(history, limit=config.max_tokens) payload = [msg.to_dict() for msg in payload_msgs]
api_response = CALL-LLM-API(config, payload)	api_response = CALL_LLM_API(config, payload)
APPEND(updated_history, reply_msg)	updated_history = history updated_history.append(reply_msg)

B. Outras partes

O pseudocódigo ficou em service.py, que é parte da configuração e conexão com as IAs, agora vamos vamos conhecer as outras partes do código:

1. config.py - Aqui é onde se encontrar o ModelConfig, onde vamos configurar a o modelo da IA, teremos *provider*, *model_name*, *temperature* etc.
2. message.py - É onde definimos a classe da mensagem, que também é um instância do pseudocódigo.
3. streamlit_app.py - O arquivo que de fato roda o programa, inicializamos nossas telas, aplicamos o streamlit e chamamos as demais funções.
4. save_conversation.py - Arquivo para auxiliar na gestão dos JSON's das conversas; listamos, salvamos, carregamos e abrimos nossas conversas. Usamos o uuid para criar um *id* único da conversa.
5. conversations - Diretório onde estão de fato os registros das conversas.

5. Detalhes

Estrutura da memória da conversa:

```
{
    "name": "Introdução a uma conversa inteligente",
    "config": {
        "provider": "gemini",
        "model_name": "gemini-2.5-flash",
        "temperature": 0.7,
        "max_tokens": 100,
        "modality": "text"
    },
    "messages": [
        {
            "role": "user",
            "content": "Olá, meu nome é jorge e matheus, com que IA falo?"
        },
        {
            "role": "assistant",
            "content": "Olá Jorge e Matheus! É um prazer conversar com vocês. Eu sou uma
        },
        {
            "role": "user",
            "content": "qual empresa que te manufaturou?"
```